

# Manual Quantum Fourier Transform and Noise Sensitivity in Period Recovery

Summer Malik, Owen Barnes, Carolina Amorim

June 2026

## Abstract

The Quantum Fourier Transform (QFT) is the interference step that makes quantum phase estimation useful for Shor’s order-finding subroutine. In this short paper, we implement the QFT and inverse QFT manually with Hadamard gates, controlled phase rotations, and final bit-reversal swaps. We validate the manual QFT against Qiskit’s built-in QFT up to five qubits, test QFT followed by inverse QFT as a round-trip identity check, and study how simulated noise changes the Fourier peaks of known-period states. Results were collected across QFT sizes from three to six qubits, three noise models, and multiple noise strengths using Qiskit Aer and Polars [3]. The manual inverse QFT was also inserted into a small Shor circuit for  $N = 15$  and  $a = 2$ , where the expected phase-estimation peaks were recovered at 0, 64, 128, and 192.

## 1 Motivation

Shor’s algorithm reduces integer factorization to order finding [1]. The quantum part of the algorithm does not directly output the factors; instead, it prepares a periodic structure and uses quantum phase estimation to extract information about the period. The inverse QFT is the final interference step that turns phase relationships into measurable peaks. This makes the QFT a useful subsystem to isolate before debugging the full Shor circuit.

The goal of this project is to make the QFT visible and testable. Rather than treating the inverse QFT as a library call, we manually construct it, validate it, and measure how different noise models affect the output distributions used for period recovery.

## 2 Manual QFT Construction

Following the standard QFT construction [2], for an  $n$ -qubit register the QFT maps a computational basis state  $|x\rangle$  to

$$\text{QFT } |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i xy/2^n} |y\rangle.$$

The implemented circuit uses a ladder of Hadamard gates and controlled phase rotations, followed by swaps that reverse the order of the qubits. The inverse QFT reverses the swap stage and applies the corresponding negative controlled phase rotations.

Two validation checks were used:

1. **Unitary comparison:** the manually implemented QFT was compared against Qiskit’s built-in QFT up to global phase.
2. **Round-trip fidelity:** basis states were evolved through QFT followed by inverse QFT, and the final state fidelity was measured against the original state.

The manual QFT matched the built-in implementation up to global phase for 2–5 qubits. The round-trip fidelity tests were approximately 1.0 for the checked basis states, confirming that the manual QFT and inverse QFT act as inverses within numerical precision.

Table 1: Manual QFT validation summary. The manual implementation was compared against Qiskit’s built-in QFT up to global phase, and QFT followed by inverse QFT was tested as a round-trip identity operation.

| Qubits | Built-in QFT Match | Round-trip Fidelity |
|--------|--------------------|---------------------|
| 2      | Yes                | $\approx 1.0$       |
| 3      | Yes                | $\approx 1.0$       |
| 4      | Yes                | $\approx 1.0$       |
| 5      | Yes                | $\approx 1.0$       |

### 3 Known-Period Experiment

Before inserting the inverse QFT into the full Shor circuit, we tested it on synthetic states with a known period. For period  $r = 4$ , the prepared state has support on equally spaced basis states:

$$|\psi\rangle = \frac{1}{\sqrt{K}} \sum_k |4k\rangle.$$

For an  $n$ -qubit register, the expected Fourier peaks occur near multiples of

$$\frac{2^n}{r}.$$

For example, when  $n = 6$  and  $r = 4$ , the expected peaks are

$$0, 16, 32, 48.$$

This experiment isolates the QFT from modular multiplication, which makes it easier to identify whether errors come from the Fourier transform stage or from the order-finding circuit around it.

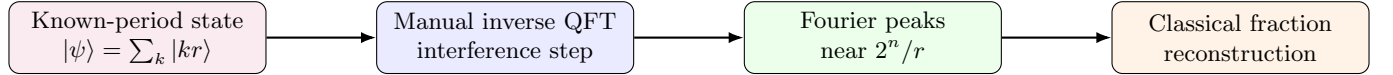


Figure 1: Known-period testing isolates the inverse QFT from modular multiplication. Because the correct period is known before simulation, changes in the output distribution can be attributed directly to the Fourier-transform stage and the applied noise model.

### 4 Noise Models and Metrics

We tested three simulated noise models:

- **Depolarizing noise:** randomizes qubit states after gates.
- **Phase damping:** reduces coherence without directly changing computational basis populations.
- **Readout error:** flips measured classical bits with a fixed probability.

Each experiment was summarized using distribution-level and algorithm-level metrics. Total variation distance measures how far the noisy output distribution is from the ideal distribution:

$$D_{\text{TV}}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)|.$$

Hellinger fidelity measures distribution similarity:

$$F_H(P, Q) = \left( \sum_x \sqrt{P(x)Q(x)} \right)^2.$$

We also measured Fourier peak mass near the expected peaks and an estimated period-success probability. The period-success probability is lower than the ideal peak mass because the zero outcome is a valid Fourier peak but does not directly recover a useful denominator for  $r$ .

## 5 Code Simulation

The codes below was built from elementary gates, starting with small circuits (2 to 5 qubits), so the gate ordering and the controlled-rotation pattern are directly visible.

Both simulations have steps in common: Hadamard gates, controlled-phase rotation gates, and swaps gates - what changes in both is mostly the order the gates are applied.

### 5.1 Regular QFT

The function processes the qubits from the highest index down to 0; each qubit receives a Hadamard followed by a cascade of controlled phase-rotations, and the whole register is reversed by Swap gates at the end.

#### 5.1.1 Step 1: Hadamard gates

The Hadamard is the entry point for each qubit. It acts on qubit  $j$ , creating an equal superposition and supplies the leading bit of phase information:

$$H|x_j\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.x_j}|1\rangle).$$

In code, one Hadamard is applied at the start of the loop for each qubit:

```
1 QFTcircuit.h(j)    # Step 1: Hadamard on qubit j
```

#### 5.1.2 Step 2: Controlled phase-rotation gates

After the Hadamard, each qubit  $j$  receives a sequence of controlled-phase gates from the qubits beneath it. The rotation angle is  $\pi/2^{(j-k)}$ , which halves as the control qubit  $k$  moves further from  $j$  ( $\pi/2$ , then  $\pi/4$ , then  $\pi/8$ , ...). These rotations write the finer binary digits of the phase onto qubit  $j$ :

```
1 for k in reversed(range(j)):    # Step 2: controlled-phase rotations
2     angle = np.pi / (2 ** (j - k))
3     QFTcircuit.cp(angle, k, j)
```

The number of these gates grows as  $0 + 1 + \dots + (n-1) = n(n-1)/2$ , the “triangle” that becomes visible in the circuit diagrams.

#### 5.1.3 Step 3: Bit-reversal Swap gates

The constructions above produces the output qubits in reversed order, so a final step of Swap gates restores the correct ordering by exchanging qubit  $i$  with its mirror  $n-1-i$ :

```
1 for i in range(n // 2):        # Step 3: bit-reversal swaps
2     QFTcircuit.swap(i, n - 1 - i)
```

#### 5.1.4 Implementation with the circuits

To complete the function, the following code builds, displays and saves each circuit as a PNG.

```
1 import numpy as np
2 from qiskit import QuantumCircuit
3 from IPython.display import display
4
5 def qft(n):
6     QFTcircuit = QuantumCircuit(n)
7
8     for j in reversed(range(n)):
9         # Step 1: Hadamards
10        QFTcircuit.h(j)
11        for k in reversed(range(j)):
12            # Step 2: Controlled-phase rotations
13            angle = np.pi / (2 ** (j - k))
14            QFTcircuit.cp(angle, k, j)
15        QFTcircuit.barrier() # Barriers are just for style
```

```

16
17     # Step 3: bit-reversal swaps
18     for i in range(n // 2):
19         QFTcircuit.swap(i, n - 1 - i)
20
21     return QFTcircuit
22
23 # 2 qubits
24 qc2 = qft(2)
25 fig = qc2.draw("mpl", plot_barriers=False)
26 fig.savefig("qft_2.png", dpi=150, bbox_inches="tight")
27 display(fig)
28
29 # 3 qubits
30 qc3 = qft(3)
31 fig = qc3.draw("mpl", plot_barriers=False)
32 fig.savefig("qft_3.png", dpi=150, bbox_inches="tight")
33 display(fig)
34
35 # 4 qubits
36 qc4 = qft(4)
37 fig = qc4.draw("mpl", plot_barriers=False)
38 fig.savefig("qft_4.png", dpi=150, bbox_inches="tight")
39 display(fig)
40
41 # 5 qubits
42 qc5 = qft(5)
43 fig = qc5.draw("mpl", fold=-1, plot_barriers=False)
44 fig.savefig("qft_5.png", dpi=150, bbox_inches="tight")
45 display(fig)

```

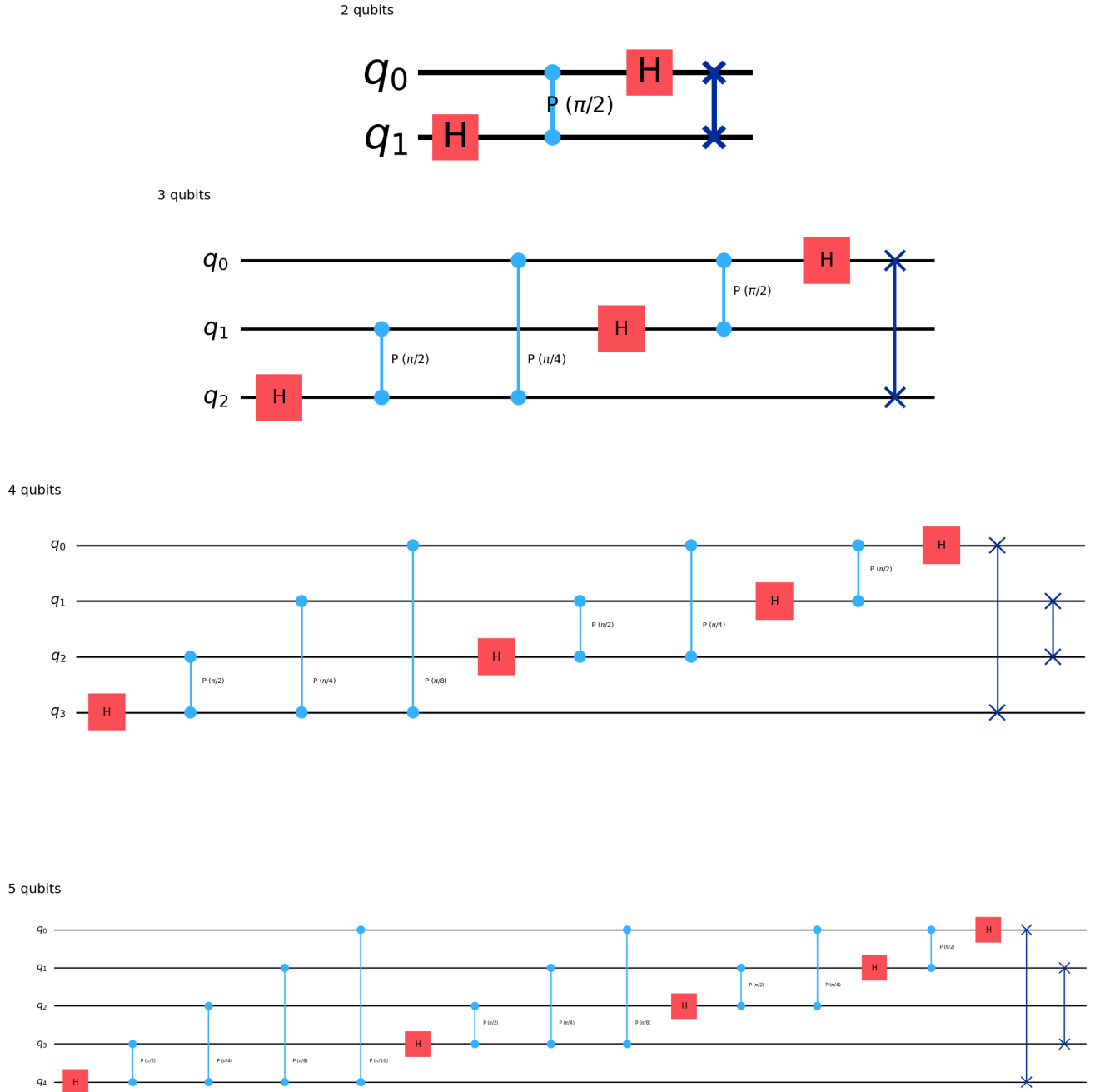


Figure 2: Manually constructed QFT circuits for 2 to 5 qubits. The triangular cascade of controlled-phase gates and the trailing SWAP layer are visible, and each larger circuit adds one finer rotation angle (the  $P(\pi/8)$  first appears at four qubits,  $P(\pi/16)$  at five).

## 5.2 Inverse QFT

The inverse QFT is the forward construction run backwards. That makes it so the transform reads a value out of the relative phases of a register and concentrates it onto a single basis state — exactly what is needed at the end of phase estimation, where the controlled operations have already encoded the answer into those phases.

### 5.2.1 Step 1: Bit-reversal Swap gates

Because the transform is reversed, the bit-reversal swaps lead the circuit rather than trailing it:

```

1 for i in range(n // 2):           # Step 1: bit-reversal swaps
2     IQFTcircuit.swap(i, n - 1 - i)

```

### 5.2.2 Step 2: Controlled phase-rotation gates

The rotations are identical to the forward case except for a negative sign on every angle,  $-\pi/2^{(j-m)}$ . This sign is the signature of the inverse transform, corresponding to the conjugated exponent  $e^{-2\pi i \dots}$  in the definition:

```

1 for m in range(j):               # Step 2: controlled-phase gates (negative angle)
2     IQFTcircuit.cp(-pi / (2 ** (j - m)), m, j)

```

### 5.2.3 Step 3: Hadamard gates

The Hadamard on each qubit is applied after that qubit's phase corrections have been removed, converting the stored phase information back into a definite 0/1 value:

```

1 IQFTcircuit.h(j)                 # Step 2.2: Hadamard on qubit j

```

### 5.2.4 Implementation with the circuits

The complete inverse-QFT function, with the same build/display/save pattern, is shown below. This routine is exactly the one substituted for the built-in inverse QFT in the Shor  $N = 21$  order-finding circuit.

```

1 from numpy import pi
2
3 def inverse_qft(n):
4     IQFTcircuit = QuantumCircuit(n)
5     # Step 1: bit-reversal swaps
6     for i in range(n // 2):
7         IQFTcircuit.swap(i, n - 1 - i)
8         IQFTcircuit.barrier() # Barriers are just for style
9
10    # Step 2: Hadamards + controlled-phase rotations
11    for j in range(n):
12        for m in range(j):
13            # Step 2.1: Controlled-Phase Gates
14            IQFTcircuit.cp(-pi / (2 ** (j - m)), m, j)
15            # Step 2.2: Hadamard Gates
16            IQFTcircuit.h(j)
17            IQFTcircuit.barrier() # Barriers are just for style
18
19    return IQFTcircuit
20
21    # 2 qubits
22    qc2 = inverse_qft(2)
23    fig = qc2.draw("mpl", plot_barriers=False)
24    fig.savefig("inverse_qft_2.png", dpi=150, bbox_inches="tight")
25    display(fig)
26
27    # 3 qubits
28    qc3 = inverse_qft(3)
29    fig = qc3.draw("mpl", plot_barriers=False)
30    fig.savefig("inverse_qft_3.png", dpi=150, bbox_inches="tight")
31    display(fig)
32
33    # 4 qubits
34    qc4 = inverse_qft(4)
35    fig = qc4.draw("mpl", plot_barriers=False)
36    fig.savefig("inverse_qft_4.png", dpi=150, bbox_inches="tight")
37    display(fig)
38
39    # 5 qubits
40    qc5 = inverse_qft(5)
41    fig = qc5.draw("mpl", fold=-1, plot_barriers=False)
42    fig.savefig("inverse_qft_5.png", dpi=150, bbox_inches="tight")
43    display(fig)

```

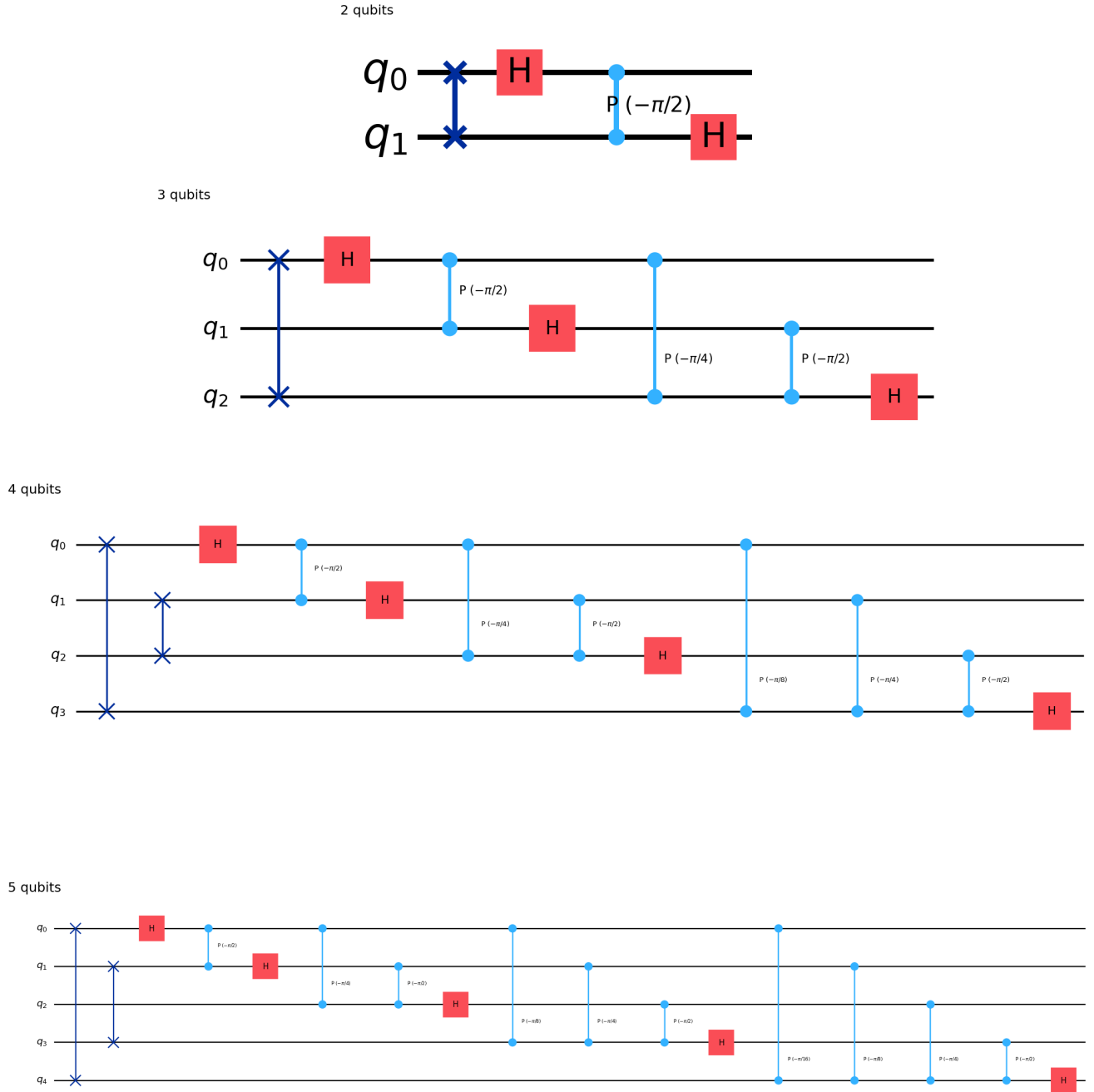


Figure 3: Manually constructed inverse QFT circuits for 2 to 5 qubits. The SWAP layer now leads the circuit and every controlled-phase angle is negated relative to the forward QFT of Figure 2.

## 6 Going Deeper into QFT Robustness

For this part, the QFT sizes and noise levels were both increased. The earlier sweep stopped at a noise level of 0.01, which was too low to visibly break the circuit — every curve stayed flat. By extending the noise range up to 0.1 (log-spaced) and testing circuits from three up to eight qubits, we can observe *how* and *when* the QFT breaks down, and how that breakdown depends on both the type of noise and the size of the circuit.

Two quantities are plotted against noise level, each faceted by noise type:

- **Period-recovery success** — whether the correct period can still be extracted from the output.
- **Total variation distance (TVD)** — how much the full output distribution differs from the ideal one.

Comparing these two is deliberate: one measures whether the circuit is still *useful*, the other measures how *damaged* it looks. They do not always agree.

## 6.1 Experiment

The experiment reuses the existing noise models and known-period circuits, sweeping over a wider grid than before. For each combination of circuit size, noise type, and noise level, the inverse QFT is applied to a known-period state, the output is measured, and both metrics are recorded. The noise levels are log-spaced so that the two orders of magnitude between  $10^{-3}$  and  $10^{-1}$  are evenly represented on the plots.

```

1  qft_sizes = [3, 4, 5, 6, 7, 8]
2
3  noise_types = ["ideal", "depolarizing", "phase_damping", "readout"]
4
5  noise_levels = [0.0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1]
6
7  results_rows = []
8  PERIOD = 4
9
10 for n in qft_sizes:
11     ideal_counts, _ = run_measured_circuit(
12         build_periodic_iqft_circuit(n, PERIOD), noise_model=None
13     )
14     ideal_p = counts_to_probability_vector(ideal_counts, n)
15     peaks = expected_peak_locations(n, PERIOD)
16
17     for noise_type in noise_types:
18         for level in noise_levels:
19             # skip nonsensical combinations
20             if noise_type == "ideal" and level != 0.0:
21                 continue
22             if noise_type != "ideal" and level == 0.0:
23                 continue
24
25             nm = get_noise_model(noise_type, level)
26             counts, _ = run_measured_circuit(
27                 build_periodic_iqft_circuit(n, PERIOD), noise_model=nm
28             )
29             p = counts_to_probability_vector(counts, n)
30
31             results_rows.append({
32                 "num_qubits": n,
33                 "noise_type": noise_type,
34                 "noise_level": level,
35                 "tv_distance": float(total_variation_distance(ideal_p, p)),
36                 "period_success_probability":
37                     estimated_period_success_from_distribution(p, n, PERIOD),
38             })
39
40 results_df = pl.DataFrame(results_rows)
41 results_df

```

## 6.2 Results

**1. Depolarizing noise is the most destructive; phase damping the least.** In both plots, depolarizing noise degrades the circuit the fastest — at the highest noise level the period-recovery success collapses toward zero and the TVD climbs above 0.9. Readout error is intermediate, and phase damping is by far the gentlest, keeping period recovery high even under strong noise. This contradicts hypotheses H2 and H3, which predicted that phase-related noise would be the most damaging, because the QFT stores its information in relative phases. The explanation is that phase damping attacks *only* the phases, and does so gently enough that the dominant peak structure survives. Depolarizing noise instead pushes the entire state toward the maximally mixed (uniform) state, attacking amplitudes and phases at once — so the channel that damages everything a little defeats the channel that damages only phases.



**2. Larger circuits break down faster.** In every panel the curves separate cleanly by size: 3 qubits is the most robust, 8 qubits the most fragile. This is expected, because a larger QFT contains more gates, so noise has more opportunities to accumulate before measurement.

**3. A degraded distribution can still recover the period.** The two plots do not move together. Under phase damping, the TVD rises to a visible level (the distribution is distorted) while period-recovery success barely falls. In other words, the output can look damaged and still contain enough information to recover the correct period. This confirms the earlier remark that distribution-level metrics such as TVD describe global degradation but do not, by themselves, determine whether order recovery remains possible.

## 7 Noise Combined Model

The previous experiments applied one noise channel at a time. Real quantum hardware does not obey those conditions: gate imperfections (depolarizing), loss of phase coherence (phase damping), and measurement errors (readout) all act together on the same circuit. This section stacks all three into a single noise model and compares it against the isolated channels.

### 7.1 Code Walkthrough

This section adds two new functions and reuses an existing one, in order to simulate all three noise channels acting at once rather than one at a time.

Making this model work required one adjustment. The periodic input state is prepared with ‘initialize’, which Aer internally expands into ‘reset’ and ‘u’ gates. Under a stacked noise model this expansion can produce a degenerate error channel and raise a ‘Kraus is empty’ error at some circuit sizes. The fix is to transpile the circuit down to explicit basis gates before simulating, so the noisy simulator never has to expand an opaque ‘initialize’ instruction itself.

```

1
2 from qiskit_aer.noise import depolarizing_error, phase_damping_error, ReadoutError
3
4 BASIS_GATES = ["h", "cp", "swap", "cx", "u", "rz", "sx", "x", "reset"]
5
6
7 def create_combined_noise_model(depol_1q, depol_2q, phase_damp, readout_p):
8     """stack depolarizing + phase damping + readout in one model."""
9
10    noise_model = NoiseModel()
11
12    # for 1 qubit gates: depolarizing & phase damping
13    noise_model.add_all_qubit_quantum_error(
14        depolarizing_error(depol_1q, 1), ["h"], warnings=False
15    )
16    noise_model.add_all_qubit_quantum_error(
17        phase_damping_error(phase_damp), ["h"], warnings=False
18    )
19
20    # for 2 qubit gates: depolarizing & phase damping on both wires
21    noise_model.add_all_qubit_quantum_error(
22        depolarizing_error(depol_2q, 2), ["cp", "swap", "cx"], warnings=False
23    )
24    pd2 = phase_damping_error(phase_damp).tensor(phase_damping_error(phase_damp))
25    noise_model.add_all_qubit_quantum_error(
26        pd2, ["cp", "swap", "cx"], warnings=False
27    )
28
29    # readout error
30    noise_model.add_all_qubit_readout_error(
31        ReadoutError([[1 - readout_p, readout_p],
32                     [readout_p, 1 - readout_p]])
33    )
34
35    return noise_model
36
37

```

```

38 def get_noise_model_extended(noise_type, noise_level):
39     """get_noise_model() plus a 'combined' option (one scalar scales all)."""
40     if noise_type == "combined":
41         return create_combined_noise_model(
42             depol_1q=noise_level,
43             depol_2q=2 * noise_level,
44             phase_damp=noise_level,
45             readout_p=noise_level / 5.0,
46         )
47     return get_noise_model(noise_type, noise_level)
48
49 # this is similar to the regular run function, but also fixes the bug
50 def run_measured_circuit_safe(circuit, noise_model=None, shots=SHOTS):
51     """like run_measured_circuit, but decomposes 'initialize' to basis
52     gates first, avoiding the 'Kraus is empty' error under stacked noise."""
53     backend = AerSimulator(noise_model=noise_model)
54     compiled = transpile(
55         circuit, backend,
56         optimization_level=1,
57         seed_transpiler=RANDOM_SEED,
58         basis_gates=BASIS_GATES, # this argument forces initialize to be expanded into basis gates so the
59         # stacked noise model doesn't crash
60     )
61     result = backend.run(
62         compiled, shots=shots, seed_simulator=RANDOM_SEED
63     ).result()
64     return result.get_counts(), compiled

```

## 7.2 Results

Combining all three noise channels into one model produces the most severe degradation of all the other models tested. At a fixed circuit size, the combined model results in the highest total variation distance and the lowest period-recovery success across the noise range. This is expected: the three channels compound - each gate now suffers both depolarizing and phase-damping errors at once, and readout error is added on top at measurement, with no cancellation between them.

One observation is which isolated channel the combined result resembles. The combined curve follows the depolarizing curve closely, sitting just slightly below it. This happens because depolarizing noise was already the most destructive channel on its own, stacking the two gentler channels on top adds only a relatively small amount of further damage. In other words, the combined model is dominated by its most damaging component: the gate-error channel sets the overall ceiling, and the milder channels contribute only second-order corrections.

In a realistic device where all noise sources are present, the dominant error channel determines how well the QFT survives. Reducing that dominant source would therefore result in the largest improvement, more than addressing the weaker channels.

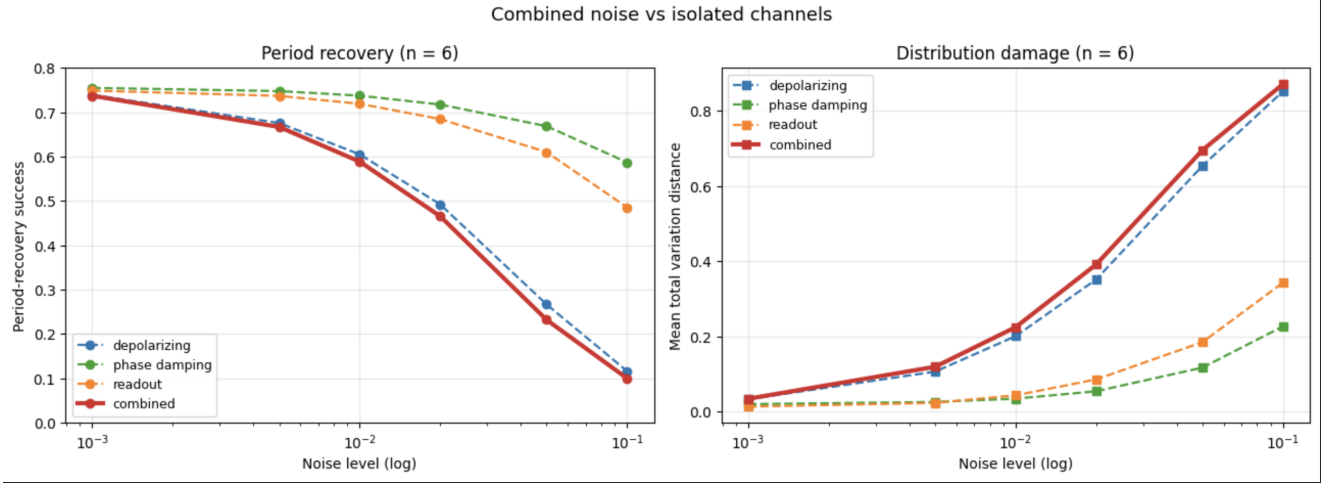


Figure 4: Comparison between the combined noise model and the three other noise models that compose it - in matters of period recovery and distribution damage. 4.

## 8 Results

Across the known-period experiments, depolarizing noise caused the clearest degradation as the number of QFT qubits increased. At  $n = 6$  and noise level 0.010, depolarizing noise produced total variation distance 0.1985, Hellinger fidelity 0.8014, peak mass 0.8389, and period-success probability 0.6096. In comparison, phase damping at the same setting was milder, with total variation distance 0.0317, peak mass 0.9766, and period-success probability 0.7407. Readout error at the same setting produced total variation distance 0.0383 and period-success probability 0.7268.

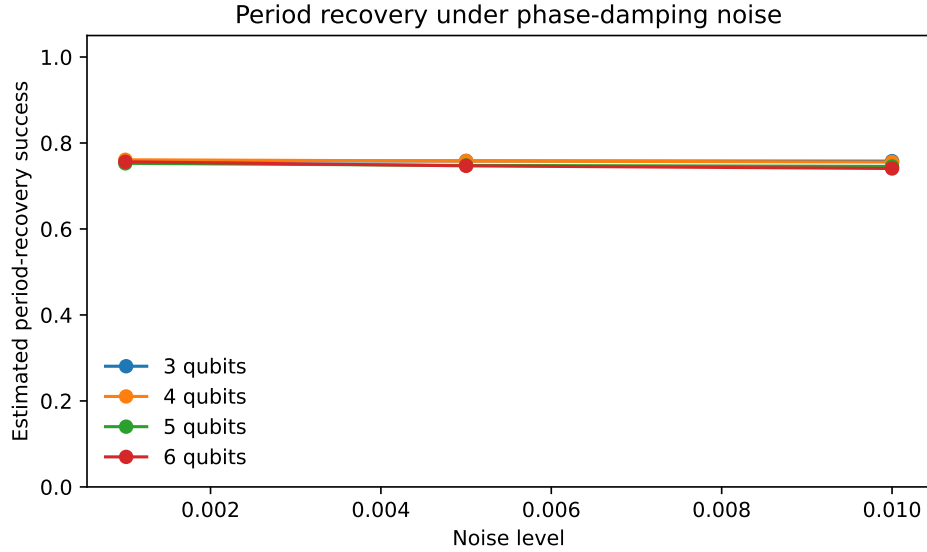


Figure 5: Estimated period-recovery success under phase-damping noise for QFT sizes from three to six qubits.

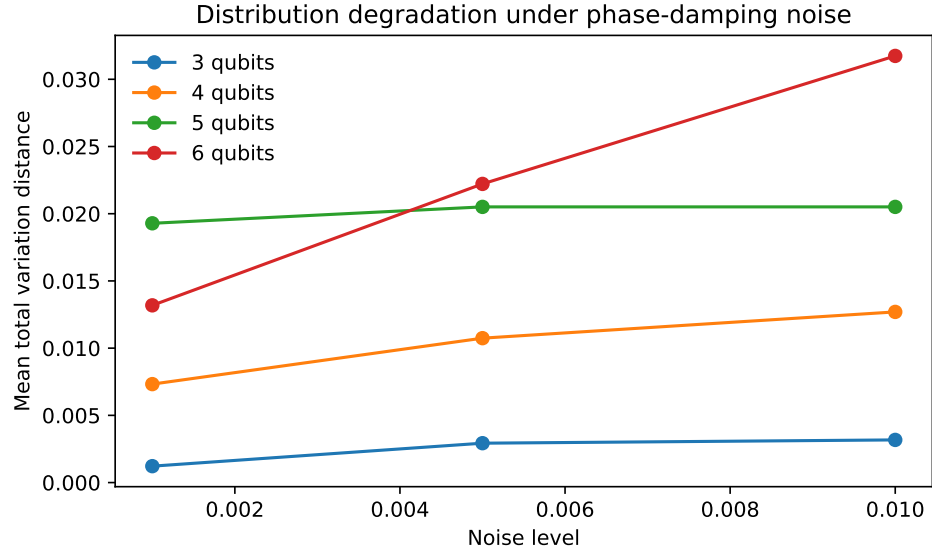


Figure 6: Total variation distance under phase-damping noise. Larger QFT registers show more visible distribution-level deviation.

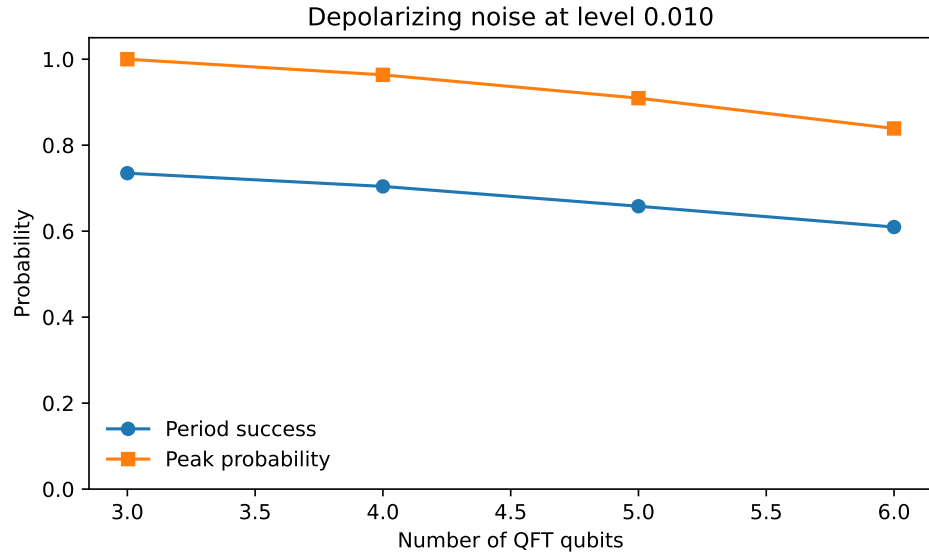


Figure 7: For depolarizing noise at level 0.010, both peak retention and period-success probability decrease as QFT size increases.

Table 2: Noise sensitivity at error level 0.010 for known-period states with  $r = 4$ . Peak mass is measured within a one-bin window of the expected Fourier peaks.

| $n$ | Noise         | TV     | Hellinger | Peak mass | Period success | Depth |
|-----|---------------|--------|-----------|-----------|----------------|-------|
| 3   | Depolarizing  | 0.0312 | 0.9687    | 1.0000    | 0.7349         | 8     |
| 3   | Phase damping | 0.0032 | 0.9978    | 1.0000    | 0.7578         | 8     |
| 3   | Readout       | 0.0105 | 0.9895    | 1.0000    | 0.7483         | 22    |
| 4   | Depolarizing  | 0.0762 | 0.9237    | 0.9636    | 0.7041         | 10    |
| 4   | Phase damping | 0.0127 | 0.9928    | 0.9958    | 0.7556         | 10    |
| 4   | Readout       | 0.0181 | 0.9819    | 0.9902    | 0.7434         | 30    |
| 5   | Depolarizing  | 0.1287 | 0.8708    | 0.9094    | 0.6580         | 12    |
| 5   | Phase damping | 0.0205 | 0.9898    | 0.9919    | 0.7456         | 12    |
| 5   | Readout       | 0.0315 | 0.9685    | 0.9802    | 0.7312         | 38    |
| 6   | Depolarizing  | 0.1985 | 0.8014    | 0.8389    | 0.6096         | 14    |
| 6   | Phase damping | 0.0317 | 0.9739    | 0.9766    | 0.7407         | 14    |
| 6   | Readout       | 0.0383 | 0.9617    | 0.9688    | 0.7268         | 46    |

## 9 Connection to Shor’s Algorithm

The manual inverse QFT was inserted into a small Shor order-finding circuit for

$$N = 15, \quad a = 2.$$

This case has order  $r = 4$ , so an eight-qubit control register should produce peaks at

$$0, 64, 128, 192.$$

The simulation recovered these four outcomes with nearly equal counts:

$$1014, 1032, 1023, 1027,$$

for the corresponding bit strings 00000000, 01000000, 10000000, and 11000000. The compiled circuit depth was 34, with operation counts including 28 controlled phase gates, 16 Hadamard gates, 15 Toffoli gates, and 8 measurements.

## 10 Discussion

The main result is that QFT degradation should not be judged only by whether the histogram changes. Some noisy distributions still retain enough mass near the expected Fourier peaks for period recovery to remain plausible. This is why algorithm-level metrics such as period-success probability are more informative than visual histogram comparison alone.

The project also shows why isolating the QFT is useful. Known-period states provide a controlled testbed where the correct Fourier peaks are known in advance. After validating this subsystem, the manual inverse QFT can be placed back into Shor’s order-finding circuit with more confidence.

## 11 Limitations and Next Steps

This work uses small simulated circuits, so it should be interpreted as a prototype rather than a scalable factoring implementation. The known-period experiments were limited to  $r = 4$  and QFT sizes up to six qubits. The Shor integration was demonstrated on the standard small case  $N = 15$ .

Next steps include testing additional periods, saving exact-versus-approximate QFT comparison data, and measuring whether approximate inverse QFT circuits can improve period recovery under noise by reducing depth. Another important next step is to run the full Shor circuit under the same noise models and compare order-recovery and factor-extraction success directly.

### 11.1 Dequantization

The dequantization of Shor’s factoring algorithm would mean to make an efficient classical algorithm that replicates its factoring/discrete-log speedup.

While it is believed to be impossible, the dequantization of Shor’s has never been ruled out and remains one of the biggest open questions in complexity theory.

### 11.1.1 Why it is belived to be impossible

The exponential speedup of Shor’s algorithm lies in the period-finding subroutine, implemented via the Quantum Fourier Transform (QFT). Given the modular exponentiation function  $f(x) = a^x \bmod N$ , the algorithm prepares a uniform superposition over exponentially many values of  $x$  and evaluates  $f$  coherently over all of them. The QFT then converts this state into one whose measurement statistics reveal the period  $r$  of  $f$  - the global structure from which the factors of  $N$  are recovered - using only a polynomial number of quantum operations.

In classic computation, no analogous shortcut is known. Although  $f$  is globally periodic, any polynomially bounded sample of its values is computationally indistinguishable from random: the period leaves no locally detectable trace. Recovering  $r$  by direct means would therefore appear to require evaluating  $f$  at exponentially many points. This intuition is reinforced empirically by decades of sustained effort motivated by cryptography, since the security of RSA and Diffie-Hellman rests on the classical hardness of factoring and the discrete logarithm. The best known classical factoring algorithm, the General Number Field Sieve, runs in sub-exponential time  $\exp(O(n^{1/3}(\log n)^{2/3}))$  for an  $n$ -bit integer, and its asymptotic exponent has not improved fundamentally since the 90s.

A structural observation supports the same conclusion. Shor’s algorithm relies on coherent interference across an exponentially large, algebraically structured state space: amplitudes corresponding to incorrect hypotheses cancel, while those consistent with the true period reinforce. The known dequantization techniques, by contrast, succeed precisely in settings where the quantum advantage is an artifact of low-rank structure combined with favorable input-access assumptions. No such structure is present in period finding, and these techniques are therefore inapplicable to this system.

### 11.1.2 Why it has never been ruled out

Despite the strong evidence against the dequantization of Shor’s, no proof excludes the possibility of an efficient classical factoring algorithm. Factoring is not known to be NP-hard, and there is good reason to believe that it’s not: the decision version of factoring lies in  $\text{NP} \cap \text{coNP}$ , so its NP-completeness would imply a collapse of the polynomial hierarchy, an outcome widely regarded as implausible. This observation cuts both ways. In one hand, it situates factoring below the NP-complete problems in presumed difficulty; on the other, it identifies factoring as a *structured* problem, and structured problems have repeatedly succumbed to unexpected classical algorithms. Primality testing is the canonical cautionary example: long believed to require randomness or super-polynomial time, it was shown to lie in P by Agrawal, Kayal, and Saxena in 2002.

Moreover, no super-polynomial classical lower bound is known for factoring, nor is one within reach of current techniques: establishing that factoring lies outside P would in particular separate P from NP. Consequently, the statement that Shor’s algorithm cannot be dequantized has the status of a conjecture rather than a theorem - and the claim that quantum computation offers an exponential advantage on a problem of practical relevance rests upon it.

### 11.1.3 Conclusion

The probability that Shor gets dequantized (efficient classical factoring found) is low — but it’s nonetheless existent. The more defensible statement is conditional: given current techniques, there is no known path, and the entire dequantization toolkit developed since 2018 is provably inapplicable to the full-rank algebraic setting Shor lives in. But absence of a lower bound means the door stays open — and either outcome would be one of the most consequential results in computer science: dequantization would break RSA overnight without any quantum hardware; a proof of impossibility would essentially prove  $P \neq \text{BQP}$ .

## 11.2 Improvements

Thirty years after Shor’s original publication, the central question surrounding quantum factoring has shifted from *feasibility* to *cost*: what are the minimal quantum resources - qubits, gates, circuit depth, and number of runs - required to factor an integer of cryptographic size? Two open fronts dominate this matter. The first concerns improvements to the algorithm itself, the most recent being Regev’s multidimensional variant of 2023; among the improvements still to be made, the most consequential and tractable is arguably closing the noise gap in Regev’s classical post-processing. The second concerns the algorithm’s complexity-theoretic status: whether its speedup could, in principle, be replicated classically - the question of dequantization - remains open, resting on conjecture rather than proof.

## Noise robustness of Regev’s algorithm

A structural asymmetry separates Regev’s multidimensional algorithm from Shor’s original when it comes to noise. Shor’s classical post-processing, based on continued fractions, tolerates per-run failure gracefully: a corrupted run is simply discarded and the circuit reruns. Regev’s post-processing, by contrast, performs lattice reduction on a *batch* of measurement outcomes, and a small fraction of corrupted samples can compromise the reconstruction of the entire batch. Although subsequent work has begun to address the robustness of the classical step, no formulation is yet known in which the post-processing degrades gracefully under a constant fraction of incorrect samples, or in which corrupted runs can be identified at a low cost. Progress here would directly affect the *practical crossover point* - the problem size at which Regev’s asymptotic advantage of  $\tilde{O}(n^{3/2})$  gates per run outweighs its batch sensitivity relative to heavily optimized implementations of Shor. Empirical characterization of how realistic noise models distort the output distribution (for instance, whether errors displace samples from the dual lattice in a structured and hence detectable manner) is a prerequisite for such algorithmic progress, and remains unexplored.

## The number-theoretic heuristic

## Lower bounds and the optimal exponent

## Space-time trade-offs for early fault-tolerant devices

## Compilation and error correction

## Directions of limited promise

## A conceptual question

# 12 Reproducibility

The experiment outputs were saved as CSV and Parquet files and organized with Polars. Each simulation row records QFT size, noise type, noise strength, circuit depth, total gate count, total variation distance, Hellinger fidelity, peak probability, and estimated period-success probability.

## References

- [1] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.
- [2] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.
- [3] Ali Javadi-Abhari et al. Quantum computing with Qiskit. arXiv preprint arXiv:2405.08810, 2024.